

ExaSentinel - Full Hackathon Build Procedure

Full Build Procedure - Neon Dye Taint Tracking + In-DB Preprocessor + UDF Loop Detector

[1. Project Overview](#)

[1.1 The Problem](#)

[1.2 The Solution: ExaSentinel](#)

[1.3 Honest Design Boundary \(state this in your pitch - it preempts the obvious question\)](#)

[2. Team Task Breakdown \(5 people\)](#)

[3. Repository Structure](#)

[4. Exasol Personal - Setup](#)

[4.1 Option 1 - Local via Docker \(fastest for a hackathon\)](#)

[4.2 Option 2 - AWS Marketplace](#)

[4.3 Option 3 - Azure Marketplace](#)

[4.4 Connectivity Check \(run this regardless of which option you picked\)](#)

[5. Demo Schema and Seed Data - `sql/01\_schema\_and\_seed\_data.sql`](#)

[6. Sensitivity Tagging, Roles, and Grants](#)

[6.1 `sql/02\_sensitivity\_tags.sql`](#)

[6.2 `sql/03\_roles\_and\_grants.sql`](#)

[7. Component A - Neon Dye Taint-Tracking Engine \(`proxy/` \)](#)

[7.1 `proxy/requirements.txt`](#)

[7.2 `proxy/registry.py` - Schema-driven sensitivity registry](#)

[7.3 `proxy/masks.py` - Masking rules per sensitivity class](#)

[7.4 `proxy/taint\_engine.py` - The core dye-propagation algorithm](#)

[7.5 `tests/test\_taint\_engine.py` - Prove the disguised-column attack is caught](#)

[8. Component A \(continued\) - FastAPI Proxy Service](#)

[8.1 `proxy/suspension\_client.py` - Checks the loop-detector blacklist](#)

[8.2 `proxy/proxy.py` - The full service](#)

[8.3 `proxy/.env.example`](#)

[9. Component B - Exasol Native In-DB Preprocessor](#)

[9.1 `sql/06\_lua\_preprocessor.sql`](#)

[9.2 Registering the preprocessor](#)

[9.3 Locking down the ability to disable it](#)

[9.4 Marking proxy statements so they aren't self-blocked](#)

[10. Component C - In-DB UDF Loop Detector](#)

[10.1 `sql/04\_query\_log\_table.sql`](#)

[10.2 `sql/05\_suspended\_agents\_table.sql`](#)

[10.3 `udf/loop\_detector\_udf.sql` - The Python UDF](#)

[10.4 Writing detected loops into the blacklist](#)

[10.5 `proxy/scheduler.py` - External trigger for the windowed scan](#)

[11. Orchestration](#)

[11.1 `docker-compose.yml`](#)

[11.2 `proxy/Dockerfile`](#)

[11.3 `scripts/init\_all.sh`](#)

[12. Integration Test Matrix](#)

[12.1 `tests/run\_demo\_matrix.py`](#)

[12.2 What each scenario proves](#)

[13. Demo Video Script \(≤ 3 minutes\)](#)

[14. README Template \(`README.md` \)](#)

[Running the Demo](#)

[Demo Video](#)

Team

15. Pitch Deck Outline (build this in [docs/pitch-deck.pdf](#) or [.pptx](#))

16. Submission Checklist

17. Known Limitations (state these proactively in your pitch)

1. Project Overview

1.1 The Problem

- AI agents (LLM copilots, autonomous SQL agents, MCP-connected tools) are increasingly given direct or semi-direct access to production databases. Two failure modes show up immediately:
- Data leakage** - a naive guardrail that blocks queries by matching column *names* (`SELECT ssn FROM ...`) is trivially defeated by an agent that renames the column in-query: `WITH secret AS (SELECT ssn AS code FROM customers) SELECT code FROM secret` . The word `ssn` never appears in the final projection, so a name-matching filter sees nothing wrong and the raw SSNs leak.
 - Runaway agents** - a buggy or looping agent can hammer the database with the same failing query dozens of times a second, degrading service for every other agent sharing the connection pool.

Both failure modes share a root cause: **enforcement that lives outside the database can be skipped**, either by an agent that gets raw credentials and connects directly, or by any request path nobody thought to instrument.

1.2 The Solution: ExaSentinel

ExaSentinel is a three-part defense-in-depth system built specifically around Exasol’s native engine capabilities:

Component	What it does	Where it runs	Defends against
A. Neon Dye Taint-Tracking Proxy	Parses every query into a full AST with <code>sqlglot</code> , tags sensitive root columns with an invisible “dye,” and tracks that dye recursively through aliases, CTEs, joins, and expressions - masking the <i>output</i> wherever the dye ends up, even if the column was renamed to hide it	Python service in front of Exasol	Disguised / renamed-column leakage attempts
B. Exasol Native In-DB Preprocessor	A <code>SQL_PREPROCESSOR_SCRIPT</code> written in Lua, running <i>inside</i> the Exasol engine on every session, that rejects direct raw-column access and cannot be turned off because <code>ALTER SESSION</code> / <code>ALTER SYSTEM</code> are revoked from the agent role	Inside the Exasol engine	The “just connect directly and skip the proxy” bypass
C. In-DB UDF Loop Detector	Every query is logged to an Exasol table; a native Python UDF runs windowed aggregation over that log to detect an agent issuing near-duplicate queries in rapid succession, and writes offending agent IDs to a blacklist table that both the proxy and the preprocessor consult	Inside the Exasol engine (UDF), triggered by an external lightweight scheduler	Runaway / looping agents degrading shared performance

Why this is Exasol-specific, not a generic pattern relabeled: Component B relies on `SQL_PREPROCESSOR_SCRIPT` , a named Exasol engine feature with no direct equivalent in Postgres/Snowflake/BigQuery (those would need extensions, triggers, or network-layer proxy-forcing - not a built-in parse-time hook). Component C relies on Exasol’s actual parallel in-memory UDF execution model to do the windowed analysis at engine speed rather than shipping logs to an external observability stack. Component A’s technique (taint analysis) is a generic CS concept, but its application to disguised-column SQL guardrails is what makes the *system* novel - and it is deliberately paired with B and C so the taint logic is enforced even when bypassed.

1.3 Honest Design Boundary (state this in your pitch - it preempts the obvious question)

The Lua preprocessor (Component B) does **not** reimplement full AST/CTE dataflow tracking - Lua's parse-time hook only gets a text/token stream, not a real syntax tree, and reimplementing `sqlglot`'s alias/CTE resolution in Lua is not realistic in a hackathon timeframe (or arguably at all, sensibly). So Component B's job is deliberately narrower than Component A: **refuse-and-redirect**, not **rewrite-in-place**. Any statement that references a sensitive column *by name* directly is hard-rejected at the engine, forcing all traffic back through the Component A proxy - which is the only credential permitted to actually read those columns. This is a *stronger* posture than trying to out-think every possible disguise in Lua: you're not playing whack-a-mole with rewrite logic, you're removing the direct path entirely.

2. Team Task Breakdown (5 people)

Role	Owns	Deliverables
1. Data/Schema Lead	Exasol Personal deployment, demo schema, sensitivity registry, permission model	Sections 4, 5, 6
2. Taint Engine Lead	The <code>sqlglot</code> -based AST taint-tracking engine	Section 7
3. Proxy/API Lead	FastAPI proxy service that wraps the taint engine and talks to Exasol	Section 8
4. In-DB Security Lead	Lua preprocessor script, permission lockdown, UDF loop detector	Sections 9, 10, 11
5. Integration/Demo Lead	Docker Compose orchestration, test matrix, demo video, README, pitch deck	Sections 12, 13, 14, 15, 16

All five roles can work in parallel after Section 4 (schema) is done, since each component only needs the schema + a stub of the others to develop against. Merge and run the full integration test matrix (Section 13) at least a day before submission.

3. Repository Structure

```
exasentinel/
├── README.md
├── LICENSE
├── docker-compose.yml
├── .env.example
├── docs/
│   ├── pitch-deck.pdf
│   ├── demo-video-link.md
│   ├── architecture-diagram.png
│   └── screenshots/
├── sql/
│   ├── 01_schema_and_seed_data.sql
│   ├── 02_sensitivity_tags.sql
│   ├── 03_roles_and_grants.sql
│   ├── 04_query_log_table.sql
│   ├── 05_suspended_agents_table.sql
│   └── 06_lua_preprocessor.sql
├── udf/
│   └── loop_detector_udf.sql
├── proxy/
│   ├── requirements.txt
│   ├── registry.py
│   ├── taint_engine.py
│   ├── masks.py
│   ├── suspension_client.py
│   ├── proxy.py
│   └── scheduler.py
├── tests/
│   ├── test_taint_engine.py
│   ├── test_integration.py
│   └── run_demo_matrix.py
└── scripts/
    ├── deploy_exasol_personal_docker.sh
    └── init_all.sh
```

4. Exasol Personal - Setup

Exasol Personal Edition is a free, single-node edition intended for local development, evaluation, and exactly this kind of hackathon use. Pick **one** of the three deployment paths below and document which one you used in your README (the rules require Exasol Personal as the primary platform, but allow Local/AWS/Azure deployment).

4.1 Option 1 - Local via Docker (fastest for a hackathon)

```
# Pull the Exasol Docker DB image (Personal/Community edition container)
docker pull exasol/docker-db:8.32.0

# Run a single-node instance
docker run --name exasol-personal \
  --detach --privileged \
  -p 8563:8563 \
  -p 2580:2580 \
  --stop-timeout 120 \
  exasol/docker-db:8.32.0 init-db --node-id 11

# Wait ~2-3 minutes for first boot, then verify it's up
docker logs -f exasol-personal
# Look for a line indicating the database is ready to accept connections
```

Default credentials for a fresh Personal instance are `sys` / `exasol` unless you set your own during `init-db` - **change this immediately** for anything beyond local dev:

```
ALTER USER sys IDENTIFIED BY 'a-strong-password-here';
```

4.2 Option 2 - AWS Marketplace

1. Search the AWS Marketplace for “Exasol” and launch the Personal/free-tier-eligible AMI into a new EC2 instance (recommend `t3.xlarge` or larger - Exasol is memory-hungry even for small workloads).
2. Open inbound security group rules for port `8563` (DB) restricted to your team’s IP range only - do not expose this publicly for a hackathon demo.
3. Note the public IP/DNS; this becomes your `EXASOL_HOST` in `.env`.

4.3 Option 3 - Azure Marketplace

1. Deploy the Exasol Personal/free-tier image from Azure Marketplace into a resource group dedicated to the hackathon.
2. Configure the Network Security Group to open port `8563` to your team’s IPs only.
3. Note the public IP/FQDN as `EXASOL_HOST`.

4.4 Connectivity Check (run this regardless of which option you picked)

```
pip install pyexasol
python3 -c "
import pyexasol
c = pyexasol.connect(dsn='localhost:8563', user='sys', password='exasol')
print(c.execute('SELECT 1').fetchall())
"
```

5. Demo Schema and Seed Data - `sql/01_schema_and_seed_data.sql`

```
CREATE SCHEMA IF NOT EXISTS DEMO;

CREATE TABLE DEMO.CUSTOMERS (
  id          INT,
  name        VARCHAR(100),
  ssn         VARCHAR(11),
  credit_card VARCHAR(20),
  email       VARCHAR(100)
);

CREATE TABLE DEMO.REFUNDS (
  id          INT,
  customer_id INT,
  refund_date DATE,
  refund_amount DECIMAL(10,2)
);

INSERT INTO DEMO.CUSTOMERS VALUES
(1, 'Alice Chen', '123-45-6789', '4111111111111111', 'alice@example.com'),
(2, 'Bob Martinez', '987-65-4321', '4222222222222222', 'bob@example.com'),
(3, 'Carla Nguyen', '555-11-2222', '4333333333333333', 'carla@example.com'),
(4, 'David Osei', '444-22-3333', '4444444444444444', 'david@example.com');

INSERT INTO DEMO.REFUNDS VALUES
(1, 1, '2026-01-15', 49.99),
(2, 2, '2026-01-20', 120.00),
(3, 1, '2026-02-02', 15.50),
(4, 3, '2026-02-10', 89.00);
```

6. Sensitivity Tagging, Roles, and Grants

6.1 `sql/02_sensitivity_tags.sql`

Column *comments* are the source of truth for sensitivity classification - both the Python taint engine and the Lua preprocessor read from this, so there is exactly one place to update when the schema changes.

```
COMMENT ON COLUMN DEMO.CUSTOMERS.ssn      IS '@pii:SSN';
COMMENT ON COLUMN DEMO.CUSTOMERS.credit_card IS '@pii:CREDIT_CARD';
COMMENT ON COLUMN DEMO.CUSTOMERS.email    IS '@pii:EMAIL';
```

6.2 `sql/03_roles_and_grants.sql`

```
-- Service account used only by the Python proxy - this is the ONLY identity
-- ever permitted to read the raw sensitive columns directly.
CREATE USER PROXY_SERVICE_USER IDENTIFIED BY 'change_me_proxy_pw';
GRANT CREATE SESSION TO PROXY_SERVICE_USER;
GRANT SELECT ON DEMO.CUSTOMERS TO PROXY_SERVICE_USER;
GRANT SELECT ON DEMO.REFUNDS TO PROXY_SERVICE_USER;
GRANT SELECT ON SYS.EXA_ALL_COLUMNS TO PROXY_SERVICE_USER;

-- Identity handed to AI agents. Deliberately weaker:
-- - no direct SELECT on the base tables containing PII
-- - cannot change session/system settings (so it can't disable the guard)
CREATE USER AI_AGENT IDENTIFIED BY 'change_me_agent_pw';
GRANT CREATE SESSION TO AI_AGENT;
REVOKE ALTER SESSION FROM AI_AGENT;
REVOKE ALTER SYSTEM FROM AI_AGENT;

-- Non-sensitive table can stay open to agents directly if you want to show
-- "not everything requires the proxy, only PII-bearing tables do"
GRANT SELECT ON DEMO.REFUNDS TO AI_AGENT;
-- Deliberately NOT granting SELECT on DEMO.CUSTOMERS to AI_AGENT.
```

7. Component A - Neon Dye Taint-Tracking Engine ([proxy/](#))

7.1 [proxy/requirements.txt](#)

```
fastapi==0.115.0
uvicorn[standard]==0.30.6
pyexasol==0.27.0
sqlglot==25.24.5
pydantic==2.9.2
apscheduler==3.10.4
python-dotenv==1.0.1
```

7.2 [proxy/registry.py](#) - Schema-driven sensitivity registry

```
"""
Loads the sensitivity registry directly from Exasol's system catalog and
column comments, so there is exactly one source of truth (the database
schema itself) rather than a hardcoded list that drifts out of sync.
"""
from __future__ import annotations
import re

TAG_PATTERN = re.compile(r"@pii:([A-Z_]+)")

def load_sensitive_columns(conn) -> dict[tuple[str, str, str], str]:
    """
    Returns { (SCHEMA, TABLE, COLUMN): SENSITIVITY_CLASS }
    Reads SYS.EXA_ALL_COLUMNS and parses the '@pii:<CLASS>' tag out of the
    column comment. This means tagging a new sensitive column is a single
    COMMENT ON COLUMN statement - no code change required anywhere.
    """
    rows = conn.execute("""
        SELECT column_schema, column_table, column_name, column_comment
        FROM SYS.EXA_ALL_COLUMNS
        WHERE column_comment IS NOT NULL
          AND column_comment LIKE '%@pii:%'
    """).fetchall()

    registry: dict[tuple[str, str, str], str] = {}
    for schema, table, col, comment in rows:
        match = TAG_PATTERN.search(comment or "")
        if match:
            key = (schema.upper(), table.upper(), col.upper())
            registry[key] = match.group(1).upper()
    return registry

def load_all_columns(conn, schema: str, table: str) -> list[str]:
    """Used to expand `SELECT *` into an explicit column list before taint
    analysis, so wildcard selects can't silently skip a tainted column."""
    rows = conn.execute(f"""
        SELECT column_name
        FROM SYS.EXA_ALL_COLUMNS
        WHERE column_schema = '{schema.upper()}'
          AND column_table = '{table.upper()}'
        ORDER BY column_ordinal_position
    """).fetchall()
    return [r[0] for r in rows]
```

7.3 [proxy/masks.py](#) - Masking rules per sensitivity class

```
SQL-level masking templates. Each class maps to a SQL expression fragment
that gets substituted for the tainted expression. __ORIG__ is a placeholder
that, if present, is replaced with the ORIGINAL expression AST (not a flat
literal) so partial-reveal masks like 'last 4 digits of a credit card' still
work correctly.

SQL_MASKS: dict[str, str] = {
    "SSN": "*****-****",
    "CREDIT_CARD": "*****-****-****-' || RIGHT(CAST(__ORIG__ AS VARCHAR(20)), 4)",
    "EMAIL": "****@**.***",
}

def mask_template_for(sensitivity_class: str) -> str:
    if sensitivity_class not in SQL_MASKS:
        # Fail closed: unknown-but-tagged sensitivity classes are fully
        # redacted rather than silently passed through.
        return "****REDACTED****"
    return SQL_MASKS[sensitivity_class]
```

```

Neon Dye taint-tracking engine.

Algorithm:
1. Seed: any column reference matching the sensitivity registry starts
   "tainted" with its class.
2. Propagate through expressions: if ANY leaf column inside an expression
   tree is tainted, the output of that expression is tainted.
3. Propagate through aliases: `ssn AS secret_id` -> secret_id is tainted.
4. Propagate through CTEs: a CTE's tainted output column taints every
   downstream reference to that CTE's column.
5. Propagate through subqueries/derived tables: each subquery is its own
   scope; its resolved output taint becomes a "virtual table" for the
   outer scope.
6. Propagate through JOINS: taint from either side of a join survives
   into the joined row set.
7. SELECT * is expanded against the live schema before analysis so
   wildcards cannot skip a tainted column.
"""
from __future__ import annotations
from dataclasses import dataclass, field
import sqlglot
import sqlglot.expressions as exp

from registry import load_all_columns
from masks import mask_template_for

@dataclass
class Scope:
    columns: dict[str, str | None] = field(default_factory=dict) # alias -> taint class or None
    tables: dict[str, str] = field(default_factory=dict) # table alias -> real table name

class TaintEngine:
    def __init__(self, registry: dict[tuple[str, str, str], str], conn=None):
        self.registry = registry # (SCHEMA, TABLE, COLUMN) -> CLASS
        self.conn = conn # used for SELECT * expansion
        self._star_cache: dict[tuple[str, str], list[str]] = {}

    # ----- Public entrypoint -----

    def analyze_and_mask(self, sql: str, dialect: str | None = None) -> tuple[str, list[str]]:
        """
        Returns (rewritten_sql, list_of_masked_output_aliases) for auditing/demo.
        """
        ast = sqlglot.parse_one(sql, read=dialect)
        self._expand_star_projections(ast)

        scopes: dict[int, Scope] = {}
        cte_scopes: dict[str, Scope] = {}
        root_scope = self._walk_select(ast, scopes, cte_scopes)

        masked_aliases = self._apply_masks(ast, root_scope)
        rewritten = ast.sql(dialect=dialect) if dialect else ast.sql()
        return rewritten, masked_aliases

    # ----- SELECT * expansion -----

```

```

def _expand_star_projections(self, ast: exp.Expression) -> None:
    for select in ast.find_all(exp.Select):
        new_expressions = []
        changed = False
        table_map = self._table_map_for(select)
        for proj in select.expressions:
            if isinstance(proj, exp.Star):
                changed = True
                for alias, real_table in table_map.items():
                    cols = self._columns_for(real_table)
                    for c in cols:
                        new_expressions.append(
                            exp.column(c, table=alias)
                        )
            else:
                new_expressions.append(proj)
        if changed:
            select.set("expressions", new_expressions)

def _columns_for(self, real_table: str) -> list[str]:
    if real_table in self._star_cache:
        return self._star_cache[real_table]
    if "." in real_table:
        schema, table = real_table.split(".", 1)
    else:
        schema, table = "DEMO", real_table
    cols = load_all_columns(self.conn, schema, table) if self.conn else []
    self._star_cache[real_table] = cols
    return cols

# ----- Scope resolution -----

def _table_map_for(self, select: exp.Select) -> dict[str, str]:
    table_map: dict[str, str] = {}
    for t in select.find_all(exp.Table):
        real_name = t.name.upper()
        alias = (t.alias or t.name).upper()
        table_map[alias] = real_name
    return table_map

def _walk_select(self, node: exp.Expression, scopes: dict, cte_scopes: dict[str, Scope]) -> Scope:
    select = node if isinstance(node, exp.Select) else node.find(exp.Select)
    if select is None:
        return Scope()

    with_ = select.args.get("with")
    if with_:
        for cte in with_.expressions:
            inner_scope = self._walk_select(cte.this, scopes, cte_scopes)
            cte_scopes[cte.alias.upper()] = inner_scope

    table_map = self._table_map_for(select)
    scope = Scope(tables=table_map)

    for proj in select.expressions:
        alias = (proj.alias_or_name or "").upper()
        taint_class = self._taint_of_expr(proj, table_map, cte_scopes)
        scope.columns[alias] = taint_class

    scopes[id(select)] = scope
    return scope

def _taint_of_expr(self, expr: exp.Expression, table_map: dict[str, str],
                   cte_scopes: dict[str, Scope]) -> str | None:
    found: str | None = None
    only_table = list(table_map.values())[0] if len(table_map) == 1 else None

    for col in expr.find_all(exp.Column):
        col_name = col.name.upper()
        tbl_alias = (col.table or "").upper()
        real_table = table_map.get(tbl_alias, tbl_alias) if tbl_alias else only_table

        if real_table is None:
            # Unqualified column with multiple tables in scope: check all
            for candidate in table_map.values():
                cls = self._registry_lookup(candidate, col_name, cte_scopes)
                if cls:
                    found = cls
            continue

        cls = self._registry_lookup(real_table, col_name, cte_scopes)
        if cls:
            found = cls

```

```

# Also check nested subqueries inside this expression
for sub in expr.find_all(exp.Subquery):
    inner_scopes: dict[int, Scope] = {}
    inner_scope = self._walk_select(sub.this, inner_scopes, cte_scopes)
    for cls in inner_scope.columns.values():
        if cls:
            found = cls

    return found

def _registry_lookup(self, real_table: str, col_name: str,
                    cte_scopes: dict[str, Scope]) -> str | None:
    # CTE reference: inherit resolved taint from the CTE's own scope
    if real_table in cte_scopes:
        return cte_scopes[real_table].columns.get(col_name)

    # Base table reference: check the registry (SCHEMA.TABLE or bare TABLE)
    for (schema, table, name), cls in self.registry.items():
        if (table == real_table or f"{schema}.{table}" == real_table) and name == col_name:
            return cls
    return None

# ----- Masking rewrite -----

def _apply_masks(self, ast: exp.Expression, root_scope: Scope) -> list[str]:
    masked: list[str] = []
    root_select = ast if isinstance(ast, exp.Select) else ast.find(exp.Select)
    if root_select is None:
        return masked

    for proj in root_select.expressions:
        alias = (proj.alias_or_name or "").upper()
        cls = root_scope.columns.get(alias)
        if not cls:
            continue
        template = mask_template_for(cls)
        mask_expr = sqlglot.parse_one(template)
        if "__ORIG__" in template:
            original = proj.this if isinstance(proj, exp.Alias) else proj
            mask_expr = sqlglot.parse_one(template.replace("__ORIG__", original.sql()))
            new_node = mask_expr.as_(alias) if alias else mask_expr
            proj.replace(new_node)
            masked.append(alias)

    return masked

```

7.5 tests/test_taint_engine.py - Prove the disguised-column attack is caught

```

import sys, os
sys.path.insert(0, os.path.join(os.path.dirname(__file__), "..", "proxy"))

from taint_engine import TaintEngine

REGISTRY = {
    ("DEMO", "CUSTOMERS", "SSN"): "SSN",
    ("DEMO", "CUSTOMERS", "CREDIT_CARD"): "CREDIT_CARD",
    ("DEMO", "CUSTOMERS", "EMAIL"): "EMAIL",
}

def test_simple_column():
    engine = TaintEngine(REGISTRY)
    sql, masked = engine.analyze_and_mask("SELECT ssn FROM DEMO.CUSTOMERS")
    assert "SSN" not in sql.upper().replace("'*_*_*_*_*'", "")
    assert "SSN" in masked

def test_alias_disguise():
    engine = TaintEngine(REGISTRY)
    sql, masked = engine.analyze_and_mask(
        "SELECT ssn AS secret_id FROM DEMO.CUSTOMERS"
    )
    assert "'*_*_*_*_*'" in sql
    assert "SECRET_ID" in masked

def test_cte_disguise():
    """The exact sneaky query from the spec: renames through a CTE."""
    engine = TaintEngine(REGISTRY)
    sql, masked = engine.analyze_and_mask(
        "WITH secret AS (SELECT ssn AS code FROM DEMO.CUSTOMERS) "
        "SELECT code FROM secret"
    )
    assert "'*_*_*_*_*'" in sql
    assert "CODE" in masked

def test_naive_name_filter_would_fail_here():
    """Demonstrates why plain name-matching guardrails miss this query."""
    sneaky_sql = (
        "WITH secret AS (SELECT ssn AS code FROM DEMO.CUSTOMERS) "
        "SELECT code FROM secret"
    )
    assert "ssn" not in sneaky_sql.lower().replace("with secret as (select ssn", "x")
    # (the word "ssn" DOES appear in the CTE definition text, so a truly naive
    # substring filter that scans raw text would catch this one - the deeper
    # point for your demo is a filter that only inspects the OUTER select's
    # column list, e.g. via regex on "SELECT <cols> FROM <last table>",
    # misses it entirely because the outer projection only says "code".)

```

8. Component A (continued) - FastAPI Proxy Service

8.1 `proxy/suspension_client.py` - Checks the loop-detector blacklist

```
"""
Consults the SUSPENDED_AGENTS table (populated by Component C, the UDF loop
detector) before letting any query through. This is what ties all three
components together: the proxy enforces both the taint mask AND the
suspension list on every request.
"""

def is_agent_suspended(conn, agent_id: str) -> bool:
    rows = conn.execute(
        "SELECT 1 FROM DEMO.SUSPENDED_AGENTS "
        f"WHERE agent_id = '{agent_id}' AND released_at IS NULL"
    ).fetchall()
    return len(rows) > 0

def log_query(conn, agent_id: str, raw_sql: str, rewritten_sql: str) -> None:
    escaped_raw = raw_sql.replace("'", "''")
    escaped_rewritten = rewritten_sql.replace("'", "''")
    conn.execute(f"""
        INSERT INTO DEMO.QUERY_LOG (agent_id, raw_sql, rewritten_sql, logged_at)
        VALUES ('{agent_id}', '{escaped_raw}', '{escaped_rewritten}', CURRENT_TIMESTAMP)
    """)
```

8.2 `proxy/proxy.py` - The full service

```

"""
ExaSentinel Proxy: the single approved path between an AI agent and the
sensitive columns in Exasol. Every request is:
1. Rejected if the agent is on the SUSPENDED_AGENTS blacklist.
2. Parsed and taint-masked by the Neon Dye engine.
3. Logged to QUERY_LOG for the UDF loop detector to analyze.
4. Executed against Exasol using the PROXY_SERVICE_USER credential
   (the agent's own weaker credential never touches the sensitive tables).
"""

import os
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import pyexasol
from dotenv import load_dotenv

from registry import load_sensitive_columns
from taint_engine import TaintEngine
from suspension_client import is_agent_suspended, log_query

load_dotenv()

EXASOL_HOST = os.getenv("EXASOL_HOST", "localhost:8563")
PROXY_USER = os.getenv("PROXY_USER", "PROXY_SERVICE_USER")
PROXY_PW = os.getenv("PROXY_PW", "change_me_proxy_pw")

app = FastAPI(title="ExaSentinel Proxy")
conn = pyexasol.connect(dsn=EXASOL_HOST, user=PROXY_USER, password=PROXY_PW)

registry = load_sensitive_columns(conn)
engine = TaintEngine(registry, conn=conn)

class QueryRequest(BaseModel):
    agent_id: str
    sql: str

class QueryResponse(BaseModel):
    rewritten_sql: str
    masked_columns: list[str]
    rows: list

@app.post("/query", response_model=QueryResponse)
def run_query(payload: QueryRequest):
    if is_agent_suspended(conn, payload.agent_id):
        raise HTTPException(
            status_code=403,
            detail=f"Agent '{payload.agent_id}' is suspended for anomalous "
                  f"query behavior. Contact an operator to release it."
        )

    try:
        rewritten_sql, masked = engine.analyze_and_mask(payload.sql)
    except Exception as e:
        raise HTTPException(status_code=400, detail=f"Could not safely parse query: {e}")

    safe_sql = rewritten_sql.rstrip(";") + " LIMIT 100"

    log_query(conn, payload.agent_id, payload.sql, safe_sql)

    try:
        rows = conn.execute(safe_sql).fetchall()
    except Exception as e:
        raise HTTPException(status_code=400, detail=f"Execution error: {e}")

    return QueryResponse(rewritten_sql=safe_sql, masked_columns=masked, rows=rows)

@app.get("/health")
def health():
    return {"status": "ok", "sensitive_columns_loaded": len(registry)}

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)

```

8.3 proxy/.env.example

```
EXASOL_HOST=localhost:8563  
PROXY_USER=PROXY_SERVICE_USER  
PROXY_PW=change_me_proxy_pw
```

9. Component B - Exasol Native In-DB Preprocessor

9.1 `sql/06_lua_preprocessor.sql`

```
CREATE OR REPLACE LUA SCRIPT SYS.SQL_GUARD (in_sql VARCHAR(2000000)) RETURNS VARCHAR(2000000) AS

-- Backstop list of sensitive column names, kept in sync with the @pii: tags
-- used by the Python taint engine (Section 6.1). This layer does name
-- matching only -- it deliberately does NOT attempt full alias/CTE
-- dataflow resolution in Lua; see Section 1.3 for why.
local SENSITIVE_COLUMNS = {
    "SSN",
    "CREDIT_CARD",
    "EMAIL"
}

-- Statements issued by the proxy's own service account carry this marker
-- comment so the preprocessor doesn't block the ONE legitimate path.
local PROXY_MARKER = "/*exasentinel-proxy*/"

function preprocessor.script(sql_text)
    local statement = sql_text.text
    local lowered = statement:lower()

    -- Allow the proxy service's own already-masked statements through
    if lowered:find(PROXY_MARKER:lower(), 1, true) then
        return sql_text
    end

    -- Always allow session bookkeeping statements (harmless, needed for
    -- clients to function) -- do NOT allow ALTER SESSION/ALTER SYSTEM though,
    -- those are blocked at the grant level (Section 6.2) as a second layer.
    if lowered:match("^%s*select%s+1%s*;%s*$") then
        return sql_text
    end

    for _, colname in ipairs(SENSITIVE_COLUMNS) do
        local lc = colname:lower()
        -- Word-boundary match so e.g. "email_verified_flag" is not a false
        -- positive trigger just because it contains "email" as a substring
        local pattern = "%f[%a]" .. lc .. "%f[%A]"
        if lowered:find(pattern) then
            error("SQL_GUARD: direct access to sensitive column '" .. colname ..
                "' is not permitted on this credential. Route requests " ..
                "through the ExaSentinel proxy service.")
        end
    end

    return sql_text
end

/
```

9.2 Registering the preprocessor

```
-- Applies engine-wide, to every session, on every statement, before parsing
ALTER SYSTEM SET SQL_PREPROCESSOR_SCRIPT = 'SYS.SQL_GUARD';
```

9.3 Locking down the ability to disable it

Already covered in `sql/03_roles_and_grants.sql` (Section 6.2):

```
REVOKE ALTER SESSION FROM AI_AGENT;
REVOKE ALTER SYSTEM FROM AI_AGENT;
```

Confirm the lockdown:

```
-- Connect as AI_AGENT and attempt to disable the guard
ALTER SESSION SET SQL_PREPROCESSOR_SCRIPT = NULL;
-- Expected: permission denied, because ALTER SESSION was revoked
```

9.4 Marking proxy statements so they aren't self-blocked

The proxy's rewritten, already-masked SQL is prefixed with the marker comment before execution. Update `proxy.py`'s `safe_sql` line:

```
safe_sql = "/*exasentinel-proxy*/ " + rewritten_sql.rstrip(";") + " LIMIT 100"
```

This is safe specifically because only `PROXY_SERVICE_USER` has the credential to reach the engine with statements that matter, and the marker only skips Component B's text scan - it does not skip Component A's masking, which already happened in Python before this string was built.

10. Component C - In-DB UDF Loop Detector

10.1 `sql/04_query_log_table.sql`

```
CREATE TABLE DEMO.QUERY_LOG (  
  id          INT IDENTITY,  
  agent_id    VARCHAR(100),  
  raw_sql     VARCHAR(2000000),  
  rewritten_sql VARCHAR(2000000),  
  logged_at   TIMESTAMP  
);
```

10.2 `sql/05_suspended_agents_table.sql`

```
CREATE TABLE DEMO.SUSPENDED_AGENTS (  
  agent_id    VARCHAR(100),  
  suspended_at TIMESTAMP,  
  reason      VARCHAR(500),  
  released_at TIMESTAMP  
);
```

10.3 `udf/loop_detector_udf.sql` - The Python UDF

```
CREATE OR REPLACE PYTHON3 SET SCRIPT DEMO.LOOP_DETECTOR (  
  window_seconds INT,  
  repeat_threshold INT  
) EMITS (agent_id VARCHAR(100), repeat_count INT, sample_sql VARCHAR(2000000)) AS  
  
import collections  
  
def run(ctx):  
    counts = collections.Counter()  
    samples = {}  
  
    # Exasol streams matching rows into ctx one at a time within this  
    # parallel UDF instance; we aggregate by agent_id within the group.  
    while True:  
        agent = ctx.agent_id  
        sql_text = ctx.raw_sql  
        counts[agent] += 1  
        if agent not in samples:  
            samples[agent] = sql_text  
        if not ctx.next():  
            break  
  
    threshold = ctx.repeat_threshold if hasattr(ctx, "repeat_threshold") else 5  
    for agent, count in counts.items():  
        if count >= threshold:  
            ctx.emit(agent, count, samples[agent])  
  
/
```

Because this is a `SET` script driven by grouped input, invoke it against a *windowed slice* of the log rather than the whole table, so “5 identical-ish calls in the last 3 seconds” is actually what gets evaluated:

```
SELECT DEMO.LOOP_DETECTOR(5, 5)  
FROM (  
  SELECT agent_id, raw_sql  
  FROM DEMO.QUERY_LOG  
  WHERE logged_at >= CURRENT_TIMESTAMP - INTERVAL '5' SECOND  
)  
GROUP BY agent_id;
```

(Adjust the `GROUP BY` /input shape based on your Exasol UDF version’s exact `SET` script grouping semantics - verify with a `SELECT COUNT(*)` -style smoke test on your Personal instance before wiring it into the scheduler, since UDF input/emit signatures are strict about column order and types.)

10.4 Writing detected loops into the blacklist

```

INSERT INTO DEMO.SUSPENDED_AGENTS (agent_id, suspended_at, reason, released_at)
SELECT agent_id,
       CURRENT_TIMESTAMP,
       'Loop detected: ' || repeat_count || ' near-duplicate queries in window',
       NULL
FROM (
  SELECT DEMO.LOOP_DETECTOR(5, 5)
  FROM (
    SELECT agent_id, raw_sql
    FROM DEMO.QUERY_LOG
    WHERE logged_at >= CURRENT_TIMESTAMP - INTERVAL '5' SECOND
  )
  GROUP BY agent_id
) detections
WHERE agent_id NOT IN (
  SELECT agent_id FROM DEMO.SUSPENDED_AGENTS WHERE released_at IS NULL
);

```

10.5 proxy/scheduler.py - External trigger for the windowed scan

Exasol Personal does not ship a built-in cron/job scheduler, so a lightweight external trigger runs this SQL on an interval. This is honest and worth stating in your pitch: **the compute happens inside Exasol** (the UDF's parallel scan), only the *trigger* is external - unlike a Grafana-style pipeline where both the compute and the trigger live outside the database.

```

"""
Runs the loop-detection query on a fixed interval and pushes results into
SUSPENDED_AGENTS. This process only ever sends a fixed, parameterized SQL
statement -- it does not accept arbitrary input, so it does not need to go
through the taint engine itself.
"""

import os
import pyexasol
from apscheduler.schedulers.blocking import BlockingScheduler
from dotenv import load_dotenv

load_dotenv()

EXASOL_HOST = os.getenv("EXASOL_HOST", "localhost:8563")
PROXY_USER = os.getenv("PROXY_USER", "PROXY_SERVICE_USER")
PROXY_PW = os.getenv("PROXY_PW", "change_me_proxy_pw")

DETECTION_SQL = """
/*exasentinel-proxy*/
INSERT INTO DEMO.SUSPENDED_AGENTS (agent_id, suspended_at, reason, released_at)
SELECT agent_id,
       CURRENT_TIMESTAMP,
       'Loop detected: ' || repeat_count || ' near-duplicate queries in window',
       NULL
FROM (
  SELECT DEMO.LOOP_DETECTOR(5, 5)
  FROM (
    SELECT agent_id, raw_sql
    FROM DEMO.QUERY_LOG
    WHERE logged_at >= CURRENT_TIMESTAMP - INTERVAL '5' SECOND
  )
  GROUP BY agent_id
) detections
WHERE agent_id NOT IN (
  SELECT agent_id FROM DEMO.SUSPENDED_AGENTS WHERE released_at IS NULL
);
"""

def scan_for_loops():
    conn = pyexasol.connect(dsn=EXASOL_HOST, user=PROXY_USER, password=PROXY_PW)
    try:
        conn.execute(DETECTION_SQL)
        print("[scheduler] loop-detection scan complete")
    finally:
        conn.close()

if __name__ == "__main__":
    scheduler = BlockingScheduler()
    scheduler.add_job(scan_for_loops, "interval", seconds=3)
    print("[scheduler] starting loop-detector every 3s")
    scheduler.start()

```

11. Orchestration

11.1 `docker-compose.yml`

```
version: "3.9"
services:
  exasol:
    image: exasol/docker-db:8.32.0
    container_name: exasol-personal
    privileged: true
    ports:
      - "8563:8563"
      - "2580:2580"
    command: init-db --node-id 11

  proxy:
    build: ./proxy
    container_name: exasentinel-proxy
    env_file: .env
    ports:
      - "8000:8000"
    depends_on:
      - exasol
    command: uvicorn proxy:app --host 0.0.0.0 --port 8000

  scheduler:
    build: ./proxy
    container_name: exasentinel-scheduler
    env_file: .env
    depends_on:
      - exasol
    command: python scheduler.py
```

11.2 `proxy/Dockerfile`

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["uvicorn", "proxy:app", "--host", "0.0.0.0", "--port", "8000"]
```

11.3 `scripts/init_all.sh`

```
#!/usr/bin/env bash
set -euo pipefail

echo "Waiting for Exasol to accept connections..."
until python3 -c "
import pyexasol
pyexasol.connect(dsn='localhost:8563', user='sys', password='exasol').close()
" 2>/dev/null; do
    sleep 5
done

echo "Applying schema..."
for f in sql/01_schema_and_seed_data.sql \
    sql/02_sensitivity_tags.sql \
    sql/03_roles_and_grants.sql \
    sql/04_query_log_table.sql \
    sql/05_suspended_agents_table.sql \
    sql/06_lua_preprocessor.sql; do
    echo "  -> $f"
    python3 -c "
import pyexasol
c = pyexasol.connect(dsn='localhost:8563', user='sys', password='exasol')
sql = open('$f').read()
for stmt in sql.split(';'):
    stmt = stmt.strip()
    if stmt and not stmt.startswith('--'):
        c.execute(stmt)
"
done

echo "Registering preprocessor..."
python3 -c "
import pyexasol
c = pyexasol.connect(dsn='localhost:8563', user='sys', password='exasol')
c.execute("ALTER SYSTEM SET SQL_PREPROCESSOR_SCRIPT = 'SYS.SQL_GUARD'")
"

echo "Deploying UDF..."
python3 -c "
import pyexasol
c = pyexasol.connect(dsn='localhost:8563', user='sys', password='exasol')
sql = open('udf/loop_detector_udf.sql').read()
c.execute(sql)
"

echo "Done. Bring up proxy + scheduler with: docker compose up proxy scheduler"
```

12. Integration Test Matrix

12.1 tests/run_demo_matrix.py

```
"""
Runs the full demo matrix end-to-end against a live stack and prints a
pass/fail table -- this doubles as your literal demo script narration.
"""

import requests
import pyexasol
import os
from dotenv import load_dotenv

load_dotenv()
PROXY_URL = "http://localhost:8000/query"
EXASOL_HOST = os.getenv("EXASOL_HOST", "localhost:8563")

def scenario_1_masked_via_proxy():
    r = requests.post(PROXY_URL, json={
        "agent_id": "demo-agent-1",
        "sql": "SELECT ssn FROM DEMO.CUSTOMERS"
    })
    ok = r.status_code == 200 and "****.*.*****" in str(r.json())
    print(f"[1] Simple masked column via proxy: {'PASS' if ok else 'FAIL'}")
    print(" ", r.json())

def scenario_2_disguised_via_cte():
    r = requests.post(PROXY_URL, json={
        "agent_id": "demo-agent-1",
        "sql": ("WITH secret AS (SELECT ssn AS code FROM DEMO.CUSTOMERS) "
              "SELECT code FROM secret")
    })
    ok = r.status_code == 200 and "****.*.*****" in str(r.json())
    print(f"[2] Disguised via CTE + alias: {'PASS' if ok else 'FAIL'}")
    print(" ", r.json())

def scenario_3_direct_connection_blocked():
    conn = pyexasol.connect(dsn=EXASOL_HOST, user="AI_AGENT", password="change_me_agent_pw")
    try:
        conn.execute("SELECT ssn FROM DEMO.CUSTOMERS")
        print("[3] Direct connection bypass: FAIL (query should have been rejected)")
    except Exception as e:
        print(f"[3] Direct connection bypass blocked as expected: PASS ({e})")

def scenario_4_cannot_disable_guard():
    conn = pyexasol.connect(dsn=EXASOL_HOST, user="AI_AGENT", password="change_me_agent_pw")
    try:
        conn.execute("ALTER SESSION SET SQL_PREPROCESSOR_SCRIPT = NULL")
        print("[4] Agent disabling guard: FAIL (should have been denied)")
    except Exception as e:
        print(f"[4] Agent cannot disable guard: PASS ({e})")

def scenario_5_loop_detection():
    for i in range(6):
        requests.post(PROXY_URL, json={
            "agent_id": "runaway-agent",
            "sql": "SELECT refund_amount FROM DEMO.REFUNDS"
        })
    import time
    time.sleep(4) # let the scheduler's window scan run
    r = requests.post(PROXY_URL, json={
        "agent_id": "runaway-agent",
        "sql": "SELECT refund_amount FROM DEMO.REFUNDS"
    })
    ok = r.status_code == 403
    print(f"[5] Loop detector suspends runaway agent: {'PASS' if ok else 'FAIL'}")

if __name__ == "__main__":
    scenario_1_masked_via_proxy()
    scenario_2_disguised_via_cte()
    scenario_3_direct_connection_blocked()
    scenario_4_cannot_disable_guard()
    scenario_5_loop_detection()
```

12.2 What each scenario proves

#	Scenario	Proves
1	Simple sensitive column via proxy	Component A works at all
2	Disguised via alias + CTE	Component A catches what name-matching filters miss
3	Direct connection to Exasol, bypassing the proxy	Component B closes the bypass
4	Agent tries to disable the guard	Grant lockdown makes Component B un-disableable
5	Agent floods identical queries	Component C detects and suspends the agent

13. Demo Video Script (≤ 3 minutes)

Time	Show	Say
0:00-0:20	Architecture diagram (proxy + Exasol + the three components)	“AI agents querying databases have two failure modes: they leak data by renaming columns, and they can loop out of control. We built ExaSentinel, three layers of defense living inside and around Exasol.”
0:20-0:50	Terminal: run scenario 1 and 2 from the matrix	“Watch this - the agent renames <code>ssn</code> to <code>code</code> through a CTE. A name-matching filter would miss this completely. Our AST taint engine tracks the dye through the alias and masks it anyway.”
0:50-1:30	Terminal: run scenario 3 and 4	“Now the agent skips our proxy entirely and connects straight to Exasol with its own credentials. The database itself rejects it - this rule lives inside the engine via <code>SQL_PREPROCESSOR_SCRIPT</code> , and the agent can’t even turn it off because we revoked that permission.”
1:30-2:10	Terminal: run scenario 5, show <code>SUSPENDED_AGENTS</code> table populate live	“A buggy agent starts looping. Exasol’s own parallel UDF engine - not an external Grafana stack - detects the pattern and suspends the agent automatically.”
2:10-2:40	Quick code scroll: <code>taint_engine.py</code> propagation function + Lua guard script	“The taint tracking is real dataflow analysis with <code>sqlglot</code> , and the in-database enforcement is native Exasol Lua - not a bolt-on proxy that can be skipped.”
2:40-3:00	README/repo structure	“All of this is deployed on Exasol Personal, fully documented, repo link and README below.”

14. README Template (README.md)

```
# ExaSentinel

Dataflow-aware PII guardrails and runaway-agent protection for AI agents
querying Exasol.

## Problem

AI agents with database access can (1) leak sensitive data by disguising
column references through aliases/CTEs, defeating simple name-matching
guardrails, and (2) degrade shared database performance by looping on
failing queries.

## Solution

Three components, described in full in `docs/architecture.md`:
- **Neon Dye Taint Engine** - AST-based dataflow tracking (`sqlglot`) that
  masks sensitive output even when columns are renamed.
- **In-DB Preprocessor** - a native Exasol `SQL_PREPROCESSOR_SCRIPT` that
  rejects direct access to sensitive columns and cannot be disabled by the
  agent role, closing the "just connect directly" bypass.
- **UDF Loop Detector** - a native Exasol Python UDF that detects
  query-flooding agents and auto-suspends them.

## Exasol Personal Usage

This project runs entirely on Exasol Personal Edition (deployed via
[Docker | AWS | Azure - state which]). Exasol Personal is used as:
- The primary data platform holding all demo data (`sql/01_schema_and_seed_data.sql`)
- The execution engine for the native Lua preprocessor (`sql/06_lua_preprocessor.sql`)
- The execution engine for the native Python UDF loop detector (`udf/loop_detector_udf.sql`)

## Setup

See `scripts/init_all.sh` and Section 4 of `docs/architecture.md` for full
step-by-step deployment instructions across all three supported paths
(Local Docker / AWS / Azure).

```bash
docker compose up -d exasol
./scripts/init_all.sh
docker compose up proxy scheduler
```

## Running the Demo

```
python tests/run_demo_matrix.py
```

## Demo Video

[link here] (≤ 3 minutes)

## Team

[names/roles per Section 2 of docs/architecture.md] ````

## 15. Pitch Deck Outline (build this in `docs/pitch-deck.pdf` or `.pptx` )

---

1. **Title** - ExaSentinel, team names, hackathon name
  2. **Problem** - one sneaky-query example on screen (the CTE disguise), one runaway-agent example
  3. **Why existing guardrails fail** - name-matching filter demo failing live or via screenshot
  4. **Architecture** - the three-layer diagram from Section 1.2's table
  5. **Why Exasol specifically** - `SQL_PREPROCESSOR_SCRIPT` and native parallel UDFs are the two features that make Components B and C possible; no bolt-on proxy could guarantee un-bypassability the way an in-engine hook can
  6. **Live demo** - or embedded video clip of the 5-scenario matrix
  7. **Honest limitations** - state Section 1.3's design boundary yourselves
  8. **What's next** - multi-node scaling, more sensitivity classes, alerting integrations
-

# 16. Submission Checklist

Requirement	Where it lives
Exasol Personal as primary platform	Section 4 - pick Local/AWS/Azure and document it
Public GitHub repo	Structure in Section 3
Source code	proxy/ , sql/ , udf/ , scripts/
Clean README	Section 14
Pitch deck (PDF/PPT)	Section 15, saved to docs/pitch-deck.pdf
Demo video ≤ 3 min or link in README	Section 13 script, link added to README
Deployment / run guide	Sections 4 and 11, plus scripts/init_all.sh
Sample data / config / screenshots	sql/01_schema_and_seed_data.sql , .env.example , docs/screenshots/

## 17. Known Limitations (state these proactively in your pitch)

---

1. **Component B is name-matching, not dataflow tracking.** It is a deliberate backstop, not a second copy of Component A - see Section 1.3.
2. **sqlglot's SQL dialect coverage.** Test every construct you plan to demo against your actual Exasol Personal instance ahead of time; not all Exasol-specific syntax round-trips perfectly through a generic dialect parser.
3. **Latency.** Benchmark the taint engine's parse+rewrite step under concurrent load before claiming any specific performance number in your pitch - AST walking is fast, but measure it, don't assert it.
4. **UDF SET script grouping semantics** vary slightly by Exasol version - validate the exact input/emit signature in Section 10.3 against your deployed version before the demo, with a small smoke test.
5. **External scheduler trigger.** Exasol Personal has no built-in job scheduler; the loop-detection *compute* is in-engine, but the *trigger* (Section 10.5) is a small external process - call this out explicitly so it reads as honest architecture rather than an overclaim.